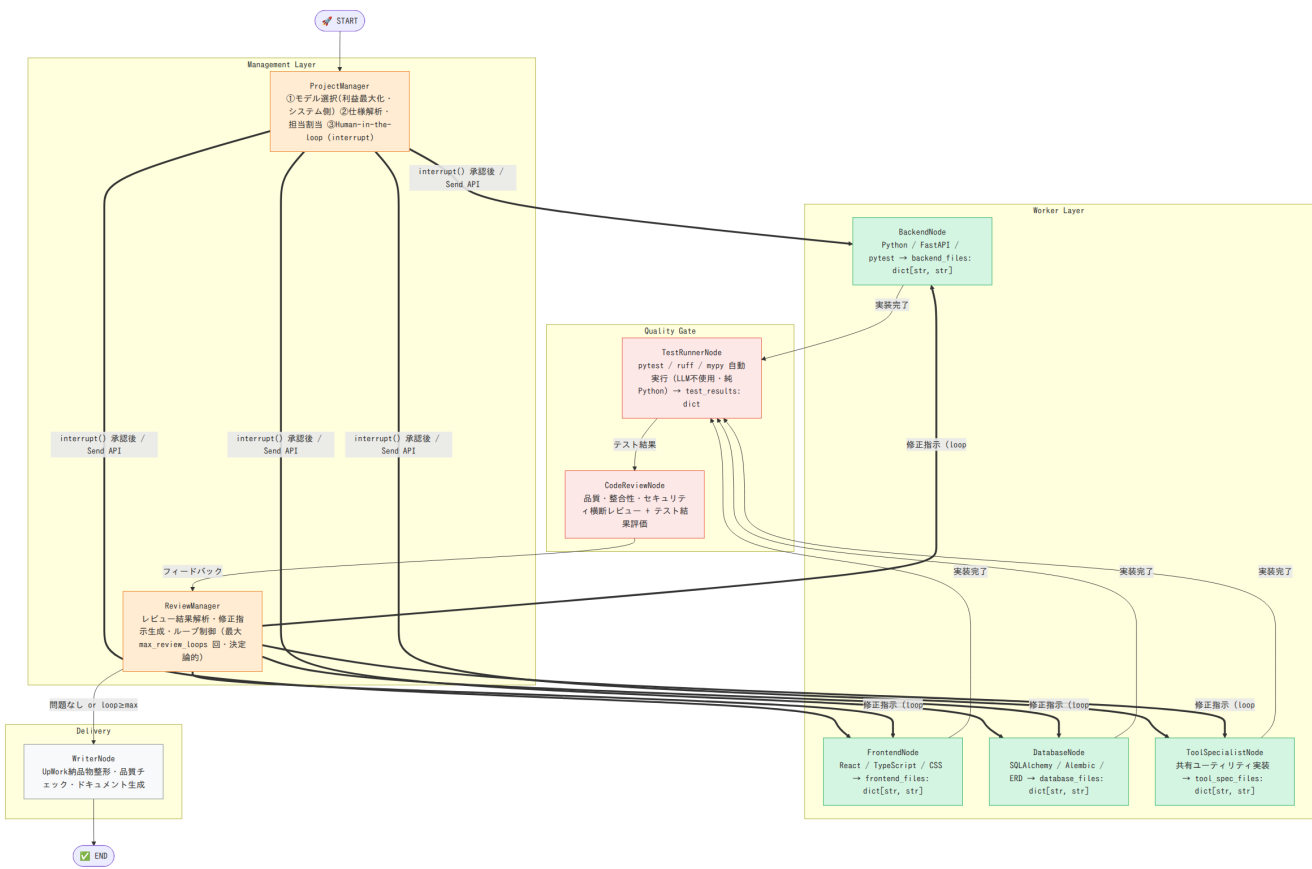


UpWork Multi-Agent System — Architecture

Version: 2.0.0 | Generated: 2026-07-01 10:53 UTC | Source: [graph/diagram_spec.py](#)

WARNING このファイルは自動生成です。直接編集しないでください。 変更は `graph/diagram spec.py` を更新してから `python tools/generate diagram.py` を実行してください。

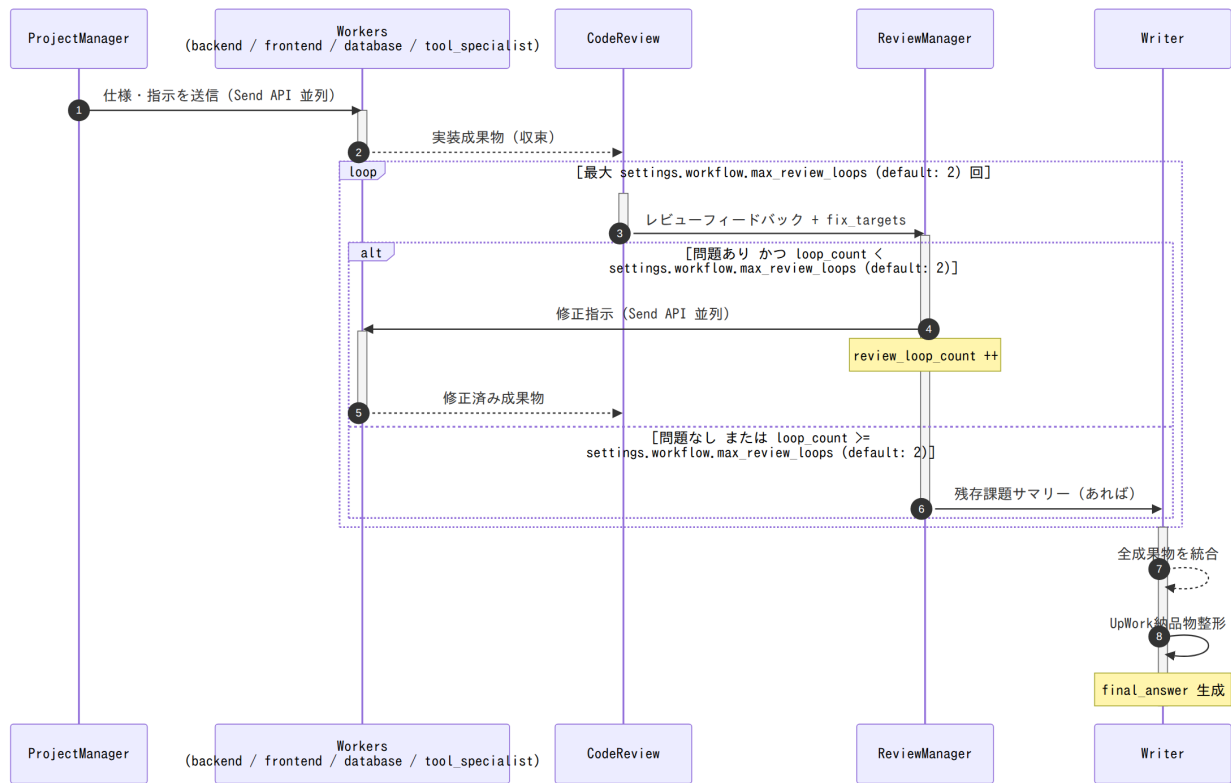
1. システム全体フロー



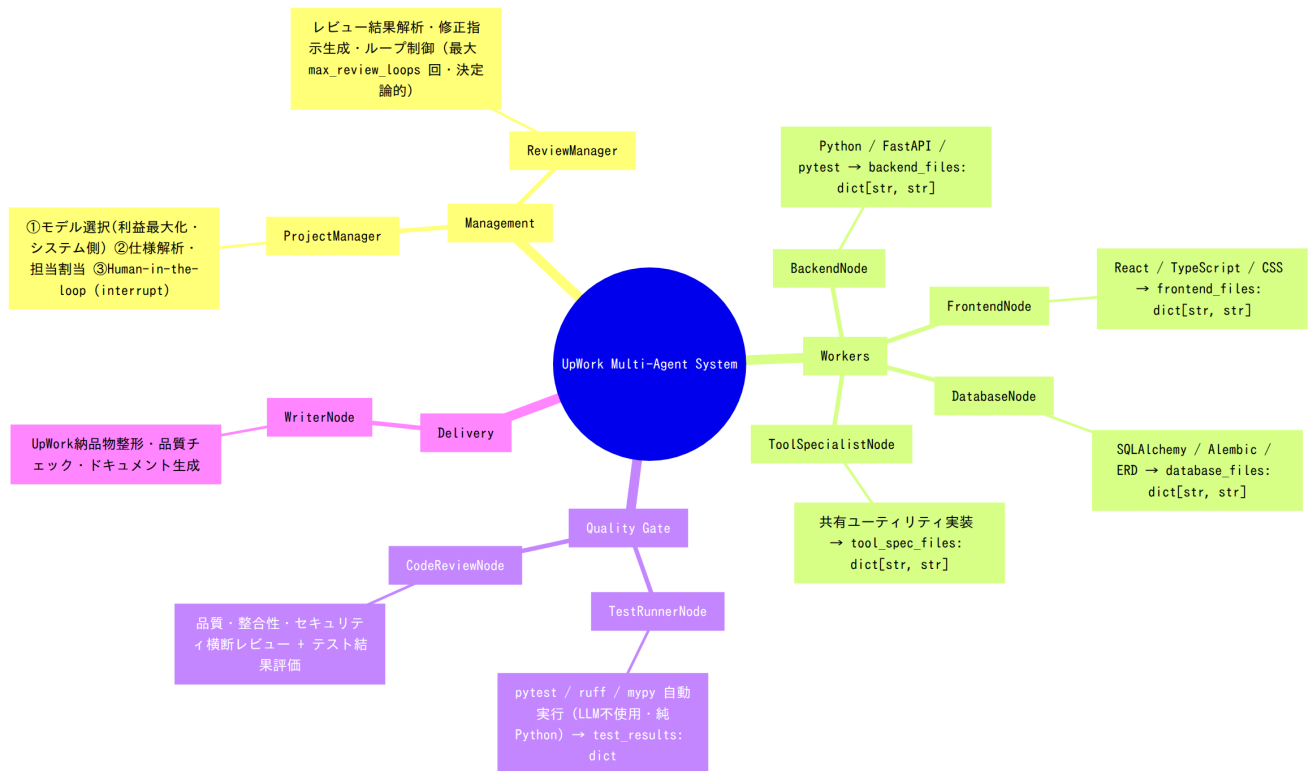
凡例 | 矢印 | 意味 | | --- | --- | | **-->** | 通常遷移 | | | **==>** | Send API 並列実行 |

2. コードレビューループ シーケンス

実装 ⇨ レビュー のループは最大 `settings.workflow.max_review_loops` (default: 2) 回 に制限されます。

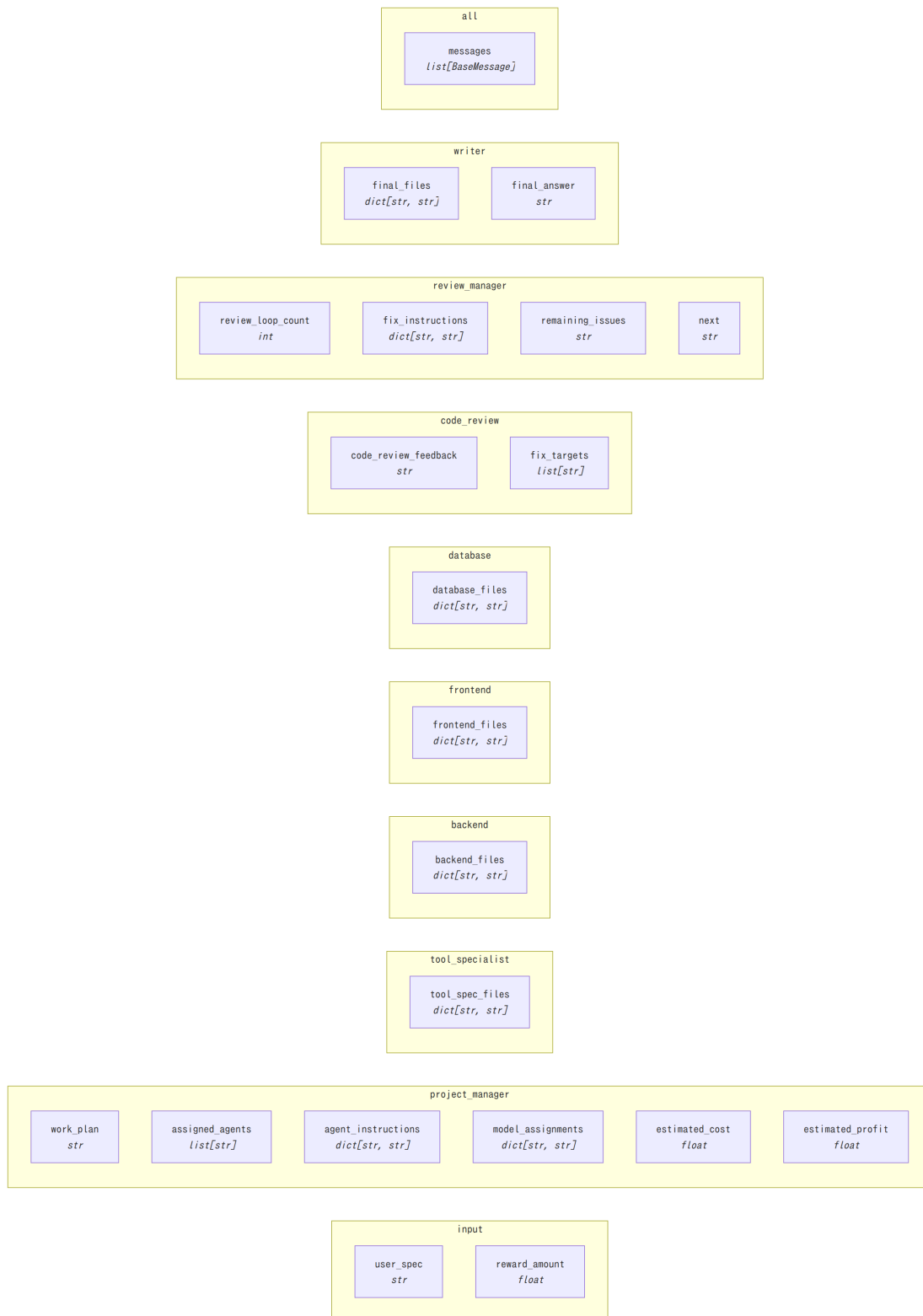


3. エージェント役割 マインドマップ



4. AgentState データフロー

各フィールドがどのエージェントによって書き込まれるかを示します。



State フィールド一覧

フィールド	型	書き込みノード	説明
messages	list[BaseMessage]	all	会話履歴 (add_messages reducer)
user_spec	str	input	UpWork クライアント仕様テキスト
work_plan	str	project_manager	作業計画書
assigned_agents	list[str]	project_manager	担当エージェント名リスト
agent_instructions	dict[str, str]	project_manager	エージェント別指示文
human_feedback	str	human	interrupt() で受け取る承認/修正指示
tool_spec_files	dict[str, str]	tool_specialist	共有ユーティリティ成果物
backend_files	dict[str, str]	backend	バックエンド成果物
frontend_files	dict[str, str]	frontend	フロントエンド成果物
database_files	dict[str, str]	database	データベース成果物
reward_amount	float	input	UpWork 報奨金 (USD) — モデル選択の基準
model_assignments	dict[str, str]	project_manager	node_name → Claude model_id (利益最大化モデル)
estimated_cost	float	project_manager	期待 API コスト (USD)
estimated_profit	float	project_manager	期待利益 (USD)
test_results	dict[str, Any]	test_runner	pytest/ruff/mypy 実行結果
code_review_feedback	str	code_review	横断コードレビュー結果
fix_targets	list[str]	code_review	修正が必要なエージェントリスト
review_loop_count	int	review_manager	レビューループ回数 (上限 max_review_loops)
fix_instructions	dict[str, str]	review_manager	エージェント別修正指示文
remaining_issues	str	review_manager	ループ上限後の残存課題
next	str	review_manager	ルーティングシグナル (fix / writer)
final_files	dict[str, str]	writer	全納品ファイル (merge済み)
final_answer	str	writer	UpWork 提出フォーマット納品物

5. ノード別 主な処理 / 出力

全ノード合計 32 処理項目

エージェント	主な処理 / 出力	数
ProjectManager	ModelSelector(決定論的割当) / WorkPlan関数コール (function calling) / interrupt()承認 / 修正指示の反映(再生成)	4
ReviewManager	escalate判定(決定論的) / ReviewDecision関数コール (function calling) / fix_instructions生成 / remaining_issues要約	4
BackendNode	write_file/read_file ツールコール / API実装+pytest+依存定義 / WorkerSubmissionで完了通知	3
FrontendNode	write_file/read_file ツールコール / HTML/CSS/TSコンポーネント+API連携 / WorkerSubmissionで完了通知	3
DatabaseNode	write_file/read_file ツールコール / ORMモデル+マイグレーション+Repository / WorkerSubmissionで完了通知	3
ToolSpecialistNode	write_file/read_file ツールコール / バリデーション/例外/ログ等の共通基盤 / WorkerSubmissionで完了通知	3
TestRunnerNode	一時ディレクトリ展開 / pytest subprocess実行 / ruff / mypy 静的解析 / 結果パース	4
CodeReviewNode	ReviewResult関数コール + read_file精査 / フルコード+テスト結果の横断評価 / OWASP観点セキュリティ確認 / fix_targets特定	4
WriterNode	全成果物マージ(純Python) / Delivery関数コール + read_file精査 / README/引き渡しドキュメント生成 / final_files + final_answer	4

6. ディレクトリ構造

```
test-BBQ/
|
|   └─ Agent_Node.py                # 基底クラス: 動的モデル選択・tool-useループ
(_run_agent)・ContextManager
|   └─ schemas.py                  # 関数スキーマ (WorkPlan/WorkerSubmission/
ReviewResult等) ★
|       └─ workspace.py            # FileWorkspace: write_file/read_file/
list_files 実行可能ツール ★
|       └─ context_manager.py      # ContextManager: メッセージトリム・アーティ
ファクト要約
|       └─ project_manager_node.py # 最上位オーケストレーター + Human-in-the-
loop
|       └─ worker_base.py          # WorkerNode基底: write_fileツールで成果物を
組み立てる共通実装
|       └─ backend_node.py         # Python/FastAPI専門 → backend_files:
dict[str,str]
|       └─ frontend_node.py       # フロントエンド専門 → frontend_files:
dict[str,str]
|       └─ database_node.py       # DB設計・実装専門 → database_files:
dict[str,str]
|       └─ tool_specialist_node.py # 共有ユーティリティ実装 → tool_spec_files:
dict[str,str]
|       └─ test_runner_node.py     # pytest/ruff/mypy 自動実行 →
test_results: dict
|       └─ code_review_node.py     # 横断コードレビュー + テスト結果評価
|       └─ review_manager_node.py # レビューループ制御 (max_review_loops)
|       └─ writer_node.py          # 納品物整形 → final_files + final_answer
|
|   └─ settings.py                # 設定 (LLM/AWS/Workflow)
|   └─ model_selector.py          # 利益最大化モデル選択 (コスト・手戻り率計算) ★
|   └─ systemMessage.py          # 全エージェントのシステムプロンプト
|
|   └─ workflow.py                # LangGraph StateGraph + MemorySaver定義
|   └─ diagram_spec.py            # 図の単一情報源 (ここを更新) ★
|
|   └─ secrets_manager.py         # AWS Secrets Manager クライアント (TTLキャッ
シュ)
|   └─ secret_keys.py            # シークレット名定数 (標準ライブラリ secrets との衝
突回避でリネーム)
|
|   └─ generate_diagram.py        # Mermaid図自動生成スクリプト ★
|
|   └─ architecture.md           # 生成されたアーキテクチャ図 (自動更新) ★
└─ pyproject.toml               # uv依存関係管理 (本番+開発) ★
└─ uv.lock                      # uv ロックファイル (自動生成・要コミット)
└─ .python-version              # Pythonバージョン固定 (3.11)
└─ .env.example                 # 非機密設定テンプレート
└─ main.py                      # エントリーポイント・DI組み立て・MemorySaver注入
```

7. 更新手順

システムにエージェント・ノード・ステートを追加したときの手順:

```
# 1. diagram_spec.py を更新 (NODES / EDGES / STATE_FIELDS)
vim graph/diagram_spec.py

# 2. Mermaid図を再生成
python tools/generate_diagram.py

# 3. コミット
git add graph/diagram_spec.py docs/architecture.md
git commit -m "docs: アーキテクチャ図更新"
```

Auto-generated by `tools/generate_diagram.py` — UpWork Multi-Agent System v2.0.0